

实验二 Linux 进程的创建与控制

1. 实验目的

- (1) 加深对进程概念的理解，明确进程与程序的区别。
- (2) 掌握 Linux 进程创建、撤销以及进程并发执行等进程管理方法。

2. 相关知识与系统调用

(1) exit()

功能：当前进程自我终止，使其进入 ZOMBIE 僵死状态，等待父进程进行善后处理。

exit()是进程正常退出的方法之一，它在头文件“stdlib.h”中声明。

(2) fork()

功能：创建子进程，执行两次返回。

返回值：-1： 失败，创建不成功

0： 在子进程的上下文中

>0：子进程的标识号，在父进程的上下文中。

fork 之后父子进程哪一个将先执行是不确定的，取决于内核调度算法。如果要求父子进程之间按照一定顺序执行，则需显式地使用进程同步。

(3) wait(0)：父进程调用后自动阻塞，直到子进程执行完撤消时才将父进程唤醒。

(4) waitpid(pid)：父进程调用后自动阻塞，只有进程号等于 pid 的子进程执行完毕才能被唤醒。

(5) getpid()：获得当前进程的 id 号。

(6) getppid()：获得当前进程的父进程的 id 号。

3. 实验程序

【程序 3_1】编写一段程序，使用系统调用 fork()创建两个子进程。此程序运行时，在系统中有一个父进程和两个子进程活动。让每一个进程在屏幕上显示一个字符：父进程显示字符“c”，子进程分别显示字符“a”和“b”。试观察屏幕上的显示结果，并分析原因。

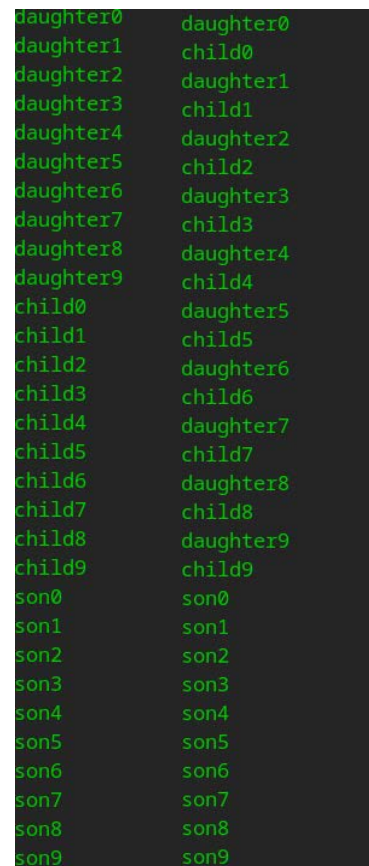
```
1. #include <stdio.h>
2. #include <unistd.h>
3. void main() {
4.     int pid, pid1;
5.     while((pid = fork( )) == -1);
6.     if(pid != 0) {
7.         while((pid1 = fork( )) == -1);
8.         if(pid1 != 0)
9.             printf("c\n");
10.        else
11.            printf("b\n");
12.    }
13.    else
14.        printf("a\n");
15. }
```

【程序运行说明】

- (1) 程序的运行结果可能是 abc，也可能是 cba，还可能是 bca、acb、cab、cba 等。
- (2) 进程创建了子进程之后，系统的调度就是随机的了，可能先调度子进程，也可能继续执行父进程。
- (3) 还可以发挥一下，用 `getpid()` 和 `getppid()` 这两个系统调用，来显示进程号。

【程序 3_2】修改已编写的程序，将每个进程的输出由单个字符改为一个字符串，再观察程序的执行时屏幕上出现的现象，并分析原因。

```
1. #include <stdio.h>
2. #include <unistd.h>
3. void main(){
4.     int pid1,pid2,i;
5.     while((pid1 = fork()) == -1);
6.     if (pid1 == 0){
7.         for (i = 0; i < 10; i++)
8.             printf("child%d\n",i);
9.     }
10.    else{
11.        while((pid2 = fork()) == -1);
12.        if(pid2 == 0){
13.            for(i = 0; i < 10; i++)
14.                printf("son%d\n",i);
15.        }
16.    }
17.    else{
18.        for(i = 0; i < 10; i++)
19.            printf("daughter%d\n",i);
20.    }
21. }
22. }
```



The screenshot shows the output of the program, displaying two columns of text. The left column contains strings from 'daughter0' to 'daughter9' and 'son0' to 'son9'. The right column contains strings from 'daughter0' to 'daughter9' and 'child0' to 'child9'. The strings are interleaved, indicating that the parent and child processes are executing concurrently. The order of execution is non-deterministic, as evidenced by the interleaving of the two columns.

【程序运行结果截图及运行说明】

由于函数 `printf()` 输出的字符串之间不会被中断，因此，字符串内部的字符顺序输出时不变。但由于进程并发执行时的调度顺序和父子进程的抢占处理机问题，输出字符串的顺序和先后随着执行的不同而发生变化。这与打印单字符的结果相同。